Final Project

RAG-Based AI Search System - Full Project Brief

LLO1	Design and present a RAG-based AI search system
LLO2	Justify system design decisions and performance outcomes
LLO3	Document and present final project code and architecture clearly
Bloom's level	Creating / Evaluating
Activity	Final presentations
Assessment	Demo and portfolio submission
Mapped to	CLO3, CLO4, PLO9, PLO10

Start this now: a runnable starter project (final_project_starter.zip) ships alongside this brief so you can have something working today and spend the remaining weeks upgrading it, rather than starting from a blank file the week before it's due.

1. What You're Building

A Retrieval-Augmented Generation (RAG) search system: a user types a question into a real interface, the system retrieves the most relevant chunks from a document collection you chose, and a language model generates an answer grounded in those chunks, with visible citations back to the source material.

This is not a chatbot with general knowledge. It must **answer from your documents** and show its work.

2. Functional Requirements

Your submission must include all of the following:

1. Document ingestion - load a real document collection (20+ documents or equivalent chunked volume) in your chosen domain: papers, manuals, articles, course notes, product docs, transcripts, or anything else you're genuinely interested in.
2. Chunking - split documents into retrievable chunks with a defensible strategy (fixed-size, sentence-aware, or section-aware).
3. Embeddings - represent chunks as vectors using a real embedding model (not TF-IDF for your final submission; see Section 5).
4. Vector search / retrieval - given a query, retrieve the top-k most similar chunks.
5. Generation - pass retrieved chunks and the query to an LLM and produce a grounded answer that cites which source(s) it used.
6. Graceful failure - when nothing relevant is found, say so instead of hallucinating an answer.
7. A working interface (required, not optional): a query input and a way to submit it; a displayed answer; a visible, expandable list of retrieved source chunks with document name and similarity score; and at least one adjustable setting (e.g. top-k, dataset, answer mode).
8. Evaluation - a small set of test queries (5–10) with a short qualitative or quantitative write-up of how well retrieval and generation performed on each.
9. Documentation - a README covering setup, architecture, and known limitations.

Non-Functional Expectations

- Modular code: ingestion, retrieval, generation, and interface should be separable pieces, not one giant script.
- Basic error handling (empty query, no results, API failure).
- Reasonable response latency for a live demo (cache what you can).

3. System Architecture

Your system moves through the following stages, from raw documents to a grounded, cited answer:

Stage	What Happens
1. Ingest & Chunk	Load documents from disk and split them into retrievable chunks.

Stage	What Happens
2. Embed	Turn each chunk into a numeric vector using an embedding model.
3. Vector Store	Index the vectors so they can be searched quickly.
4. Retrieve	Embed the user's query and retrieve the top-k most similar chunks.
5. Generate (LLM)	Pass the query and retrieved chunks to an LLM to produce a grounded, cited answer.
6. Interface	Take the query in and display the answer and sources back to the user.

This is the same shape as the Week 14 lab's pipeline (features, vectorize, profile, similarity, rank), with two additions: embeddings replace TF-IDF, and a generation step turns retrieved text into a written answer.

4. Interface Requirements

A minimal, functional layout is enough; polish is a bonus, not a requirement. At minimum your interface needs:

- Header - project/system name
- Query box + submit control
- Answer panel - the generated response
- Sources panel - each retrieved chunk shown with source document name, similarity score, and chunk text (collapsible/expandable is fine)
- Sidebar or settings area - at minimum, number of chunks to retrieve (top_k); optionally a dataset selector, answer mode, or latency display

Any of these satisfy the interface requirement: Streamlit, Gradio, or a Flask/FastAPI + simple HTML/JS front end. Pick whichever you're fastest in; the starter project uses Streamlit because it gets you a working UI in one file.

5. Suggested Tech Stack

Layer	Simple Option	Stronger Option
Interface	Streamlit or Gradio	Flask/FastAPI + custom HTML/JS
Embeddings	sentence-transformers (local, free, no API key)	OpenAI / Anthropic / Cohere embeddings API
Vector store	In-memory cosine similarity (fine under a few thousand chunks)	FAISS or Chroma
Generation LLM	Claude or GPT via API	Same, with deeper prompt-engineering / evaluation
Language	Python throughout	Python throughout

You are free to substitute any of these; the requirement is the architecture, not a specific library.

6. Milestones

Adjust these to your actual number of remaining weeks; the point is the order, not the exact dates.

Checkpoint	Target	What "Done" Looks Like
0 - This week	Now	Starter project running locally; domain/dataset chosen
1	Next week	Your own documents ingested and chunked; chunk count printed and sane
2	Next week	Real embeddings + retrieval working end-to-end (query to top-k chunks)
3	Next week	LLM generation wired in, answers cite sources; interface functional
4	Next 2 weeks	Evaluation write-up done; interface polished; README complete
Final	Presentation day 29 July 2026	Live demo + presentation (LLO1-3) + portfolio submission

Start This Week

1. Unzip and run final_project_starter.zip; confirm the demo works before changing anything.
2. Pick your document domain and swap data/sample_docs/ for real content.
3. Re-run the app with your own data using the existing TF-IDF pipeline (same mechanics as the Week 14 lab) so you have an early, working baseline.
4. Only after that baseline works, begin the embeddings upgrade (Checkpoint 2).

7. Deliverables

- Code repository (all four architecture layers present and separable)
- A working, running interface (deployed locally is fine; hosted is a bonus)
- Written evaluation (5–10 test queries + short discussion)
- README documenting setup, architecture, and design decisions
- Live demo + presentation covering LLO1–LLO3

8. From Today's Lab to This Project

Week 14 Lab	Final Project
Hardcoded movie catalog	Your own document collection
TF-IDF vectors	Real embeddings
In-memory cosine similarity	Same idea, optionally a real vector database
Printed top-5 list	Full interface with sources + generated answer
No generation step	LLM call grounded in retrieved chunks

You are not starting from zero in three weeks. You are upgrading something that already works, one layer at a time, starting today.